

```
import seaborn as sns
df=sns.load_dataset("titanic")
df
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class
0	0	3	male	22.0	1	0	7.2500	S	Third
1	1	1	female	38.0	1	0	71.2833	C	First w
2	1	3	female	26.0	0	0	7.9250	S	Third w
3	1	1	female	35.0	1	0	53.1000	S	First w
4	0	3	male	35.0	0	0	8.0500	S	Third
...	...	...	...	...	...	...	...	...	...
886	0	2	male	27.0	0	0	13.0000	S	Second
887	1	1	female	19.0	0	0	30.0000	S	First w
888	0	3	female	NaN	1	2	23.4500	S	Third w
889	1	1	male	26.0	0	0	30.0000	C	First
890	0	3	male	32.0	0	0	7.7500	Q	Third

891 rows × 15 columns

Next steps:

[Generate code with df](#)

[New interactive sheet](#)

```
df.drop(["adult_male", "alone"], axis=1, inplace=True)
```

```
df.isnull().sum()
```

	0
survived	0
pclass	0
sex	0
age	177
sibsp	0
parch	0
fare	0
embarked	2
class	0
who	0
deck	688
embark_town	2
alive	0

dtype: int64

Double-click (or enter) to edit

```
df["age"]=df["age"].fillna(df["age"].mean())
df
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	cla
0	0	3	male	29.699118	1	0	7.2500	S	Th
1	1	1	female	29.699118	1	0	71.2833	C	Fi
2	1	3	female	29.699118	0	0	7.9250	S	Th
3	1	1	female	29.699118	1	0	53.1000	S	Fi
4	0	3	male	29.699118	0	0	8.0500	S	Th
...	...	...	...	...	...	...	...	...	
886	0	2	male	29.699118	0	0	13.0000	S	Seco
887	1	1	female	29.699118	0	0	30.0000	S	Fi
888	0	3	female	29.699118	1	2	23.4500	S	Th
889	1	1	male	29.699118	0	0	30.0000	C	Fi
890	0	3	male	29.699118	0	0	7.7500	Q	Th

891 rows × 13 columns

Next steps:

[Generate code with df](#)[New interactive sheet](#)

```
df['sex']=df['sex'].fillna(df['sex'].mode()[0])
df
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class
0	0	3	male	29.699118	1	0	7.2500	S	Third
1	1	1	female	29.699118	1	0	71.2833	C	First
2	1	3	female	29.699118	0	0	7.9250	S	Third
3	1	1	female	29.699118	1	0	53.1000	S	First
4	0	3	male	29.699118	0	0	8.0500	S	Third
...	...	...	...	...	...	...	...	...	...
886	0	2	male	29.699118	0	0	13.0000	S	Second
887	1	1	female	29.699118	0	0	30.0000	S	First
888	0	3	female	29.699118	1	2	23.4500	S	Third
889	1	1	male	29.699118	0	0	30.0000	C	First
890	0	3	male	29.699118	0	0	7.7500	Q	Third

891 rows × 13 columns

Next steps:

[Generate code with df](#)[New interactive sheet](#)Start coding or [generate](#) with AI.

```
import pandas as pd
df=pd.get_dummies(df,drop_first=True)
df
```

	survived	pclass	age	sibsp	parch	fare	sex_male	embarked_Q
0	0	3	29.699118	1	0	7.2500	True	False
1	1	1	29.699118	1	0	71.2833	False	False
2	1	3	29.699118	0	0	7.9250	False	False
3	1	1	29.699118	1	0	53.1000	False	False
4	0	3	29.699118	0	0	8.0500	True	False
...	...	...	...	...	...	...	...	...
886	0	2	29.699118	0	0	13.0000	True	False
887	1	1	29.699118	0	0	30.0000	False	False
888	0	3	29.699118	1	2	23.4500	False	False
889	1	1	29.699118	0	0	30.0000	True	False
890	0	3	29.699118	0	0	7.7500	True	True

891 rows × 22 columns

```
x=df.drop('survived',axis=1)
y=df['survived']
```

x

	pclass	age	sibsp	parch	fare	sex_male	embarked_Q	embarked_S
0	3	29.699118	1	0	7.2500	True	False	True
1	1	29.699118	1	0	71.2833	False	False	False
2	3	29.699118	0	0	7.9250	False	False	True
3	1	29.699118	1	0	53.1000	False	False	True
4	3	29.699118	0	0	8.0500	True	False	True
...	...	...	...	...	...	...	...	...
886	2	29.699118	0	0	13.0000	True	False	True
887	1	29.699118	0	0	30.0000	False	False	True
888	3	29.699118	1	2	23.4500	False	False	True
889	1	29.699118	0	0	30.0000	True	False	False
890	3	29.699118	0	0	7.7500	True	True	False

891 rows × 21 columns

```
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.25,random_state=42)
x
```

	pclass	age	sibsp	parch	fare	sex_male	embarked_Q	embarked_S
0	3	29.699118	1	0	7.2500	True	False	True
1	1	29.699118	1	0	71.2833	False	False	False
2	3	29.699118	0	0	7.9250	False	False	True
3	1	29.699118	1	0	53.1000	False	False	True
4	3	29.699118	0	0	8.0500	True	False	True
...	...	...	...	...	...	...	...	...
886	2	29.699118	0	0	13.0000	True	False	True
887	1	29.699118	0	0	30.0000	False	False	True
888	3	29.699118	1	2	23.4500	False	False	True
889	1	29.699118	0	0	30.0000	True	False	False
890	3	29.699118	0	0	7.7500	True	True	False

891 rows × 21 columns

```
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.25,random_state=42)
y
```

survived	
0	0
1	1
2	1
3	1
4	0
...	...
886	0
887	1
888	0
889	1
890	0

891 rows × 1 columns

dtype: int64

```
from sklearn.linear_model import LogisticRegression
model=LogisticRegression()
model.fit(x_train,y_train)
```

/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:465
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```
n_iter_i = _check_optimize_result(
```

▼ LogisticRegression ⓘ ?

```
LogisticRegression()
```

```
y_pred=model.predict(x_test)
y_pred
```

```
array([1, 0, 0, 1, 1, 1, 1, 0, 1, 1, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0, 1,
       1, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 1, 0, 1, 1, 1, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 1, 1, 0, 1, 0, 1, 0, 1, 1, 1, 0, 1, 1, 0, 0, 0,
       0, 0, 0, 1, 0, 1, 1, 1, 0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 0, 0, 1, 1,
       0, 0, 1, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 1, 1,
       1, 0, 1, 1, 0, 0, 0, 0, 1, 0, 0, 1, 1, 1, 0, 1, 1, 0, 0, 0, 1, 1,
       0, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 1,
       0, 0, 1, 1, 1, 0, 0, 1, 0, 1, 0, 0, 0, 1, 0, 0, 1, 1, 1, 0, 0, 0,
       1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 1, 1, 1, 1, 0,
       1, 0, 0, 0, 1, 1, 1, 1, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 1, 0,
       0, 0, 0])
```

```
y_prob=model.predict_proba(x_test)
y_prob
```

```
[0.93517391, 0.06482609],  
[0.99182354, 0.00817646]])
```

```
from sklearn.metrics import accuracy_score  
accuracy_score(y_test,y_pred)
```

```
1.0
```

```
from sklearn.metrics import confusion_matrix  
conf_matrix=confusion_matrix(y_test,y_pred)  
conf_matrix
```

```
array([[134,  0],  
       [ 0,  89]])
```

```
from sklearn.metrics import classification_report  
print(classification_report(y_test,y_pred))
```

```
precision    recall  f1-score   support
```